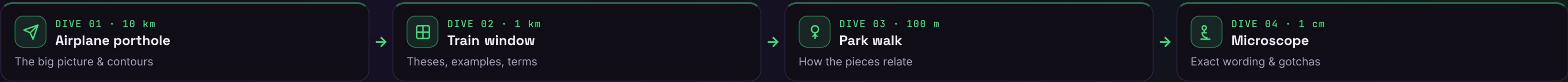


Iterator

Provide sequential access to an aggregate’s elements without exposing its underlying representation.



 Airplane porthole

DIVE 01 · ORIENTATION

▼ 10 km

It factors traversal out of the collection — the cursor remembers “where am I”, so callers walk elements without ever touching the internal structure.

THE CONTOURS

• Decouples traversal from storage
— array, tree or list, one uniform walk

• Cursor owns the position
— many independent passes over one collection

• Lazy by nature
— pull one element at a time, even an infinite stream

USE IT WHEN — GoF applicability

 Access an aggregate's contents
without exposing its internal representation

 Support multiple, simultaneous traversals
or a uniform interface across different collections

Behavioral family: Iterator · Observer · Strategy · Command · Visitor — Iterator streams elements; Visitor adds operations over an already-walked structure.

↓ DIVE DEEPER

 Train window

DIVE 02 · KEY OBJECTS

▼ 1 km

THE THREE PARTS

Iterator
the cursor interface — hasNext() / next() / current

ConcreteIterator
holds the traversal position over one aggregate

Aggregate
createIterator() — the collection that yields cursors

IN THE WILD

 Symbol.iterator / for...of
JS iteration protocol — generators are function*

 java.util.Iterator
hasNext()/next()/remove() across the collections

 IEnumerator / IEnumerable
C# foreach; LINQ is lazy iterators

 Readable streams
Node Symbol.asyncIterator — for await...of chunks

```
<> Client → Aggregate.createIterator() → Iterator.hasNext()/next() → element
```

GoF intent: “Provide a way to access the elements of an aggregate object sequentially without exposing its underlying representation.”

↓ DIVE DEEPER

 Park walk

DIVE 03 · CONNECTIONS

▼ 100 m

Collection
array · tree · list

→

createIterator()
hand out a cursor

→

hasNext() ? next()
advance the position

→

✓ element
yielded to caller

WHY IT FOLLOWS

→ Cursor holds position ⇒ the collection stays stateless about who is walking it

→ Uniform interface ⇒ one loop works over array, tree or remote page

→ Pull, not push ⇒ elements are produced lazily, on demand, one at a time

Composite is often walked by an Iterator · Factory Method creates the right iterator · Visitor pairs with it to run ops per element · Memento can snapshot a robust cursor.

STRUCTURE · UML

```
classDiagram
    class IteratorT {
        - pos: number
        + hasNext(): boolean
        + next(): T
        + current(): T
    }
    class ConcreteIterator {
        <<private>>
    }
    class Aggregate {
        + createIterator()
    }
    IteratorT --> ConcreteIterator
    IteratorT --> Aggregate
```

ConcreteIterator keeps -pos over the aggregate; next() returns the element and advances.

WHAT IT BUYS YOU

✓ Traversal variants — forward, reverse, DFS/BFS — swap the iterator, not the collection

✓ Simplifies the aggregate — no traversal methods crowd the collection's interface

✓ Many passes at once — each iterator has its own cursor — concurrent walks

✓ Uniform access — one for...of over wildly different containers

✓ Lazy & bounded memory — stream huge or infinite sequences without materializing

↓ DIVE DEEPER

 Microscope

DIVE 04 · DETAILS

▼ 1 cm

Definition: “Provide a way to access the elements of an aggregate object sequentially without exposing its underlying representation.”
— GoF, “Design Patterns”, 1994

THE CODE

```
class Range implements Iterable<number> {
  constructor(private lo: number,
    private hi: number) {}
  *[Symbol.iterator]() {
    for (let i = this.lo; i < this.hi; i++)
      yield i;      // lazy, one at a time
  }
}
for (const n of new Range(0, 3)) use(n);
// generator IS the ConcreteIterator here
```

COMPARE THE ALTERNATIVES

Property	Iterator	for-loop	Generator	Stream
Lazy / on demand	✓	✗	✓	✓
Hides structure	✓	✗	✓	✓
Infinite sequence	✓	✗	✓	~
Async I/O	~	✗	~	✓
Boilerplate	~	✓	✓	~

Best fit → Iterator: custom traversal · for-loop: dense arrays · Generator: lazy state · Stream: async I/O. (“~” = partial.)

INTERVIEW PITFALLS

Q “External vs internal iterator?”
External: the client calls next() and controls the loop.
Internal: the collection runs the loop and calls you back (forEach).

Q “What happens if you mutate mid-iteration?”
Fail-fast iterators throw (ConcurrentModificationException);
snapshot iterators walk a copy. JS just sees the live array.

Q “Is a generator an iterator?”
Yes — calling a function* returns an iterator (and iterable)
with next(); it's one-shot until you call the factory again.

Q “Why lazy iteration?”
You process one element at a time, so you can handle
infinite or huge sequences in constant memory.

VARIANTS

External
caller drives next() — full control, classic GoF

Internal
collection drives, you pass a fn — forEach/map

Generator
function* yields — state machine for free

Lazy / infinite
compute on demand — Fibonacci, paged feeds

Robust
fail-fast vs snapshot — survive mutation

Async
Symbol.asyncIterator — for await over I/O

NUANCES & GOTCHAS

• Invalidation on mutation — editing the collection mid-walk breaks the cursor; Java throws ConcurrentModificationException (fail-fast), some snapshot instead

• One-shot vs reusable — a bare generator object is exhausted after one pass; an Iterable mints a fresh iterator each for...of

• Lazy evaluation — nothing is computed until pulled, so memory stays flat over huge or infinite sequences

• Sync vs async — Symbol.iterator → for...of; Symbol.asyncIterator → for await...of for streamed I/O

• Statefulness — the cursor carries position; sharing one iterator across consumers corrupts the walk

• Early-break cleanup — return()/finally runs on break or throw so resources (files, sockets) get released

WATCH OUT

 Anti-pattern: A plain array you'll fully load and index is simpler with a for-loop — reach for a custom iterator only when traversal is non-trivial, lazy, or the structure must stay hidden.

 Modern take: Generators and Symbol.iterator make this almost free in JS/Python/C#; you rarely hand-write hasNext()/next() — and async iterators extend the same protocol to streams.

• Vasyi Krupka · Senior Fullstack Engineer · learn-by-diving series

Source: GoF “Design Patterns” (1994) v1